Greedy Method:

→ Divide-and-conquer technique is applicable only for problems, which can be divisible. There exist some problems which can't be divisible.

Example: knapsack problem

→ This is the drawback of divide-and-conquer technique, which is rectified in greedy method.

AIM: To calculate optimal solution

→ Most of the problems have n inputs and require us to obtain a subset that satisfies some constrain.

→ Any subset that satisfies these constraints is called feasible solution

→ We need to find a feasible solution that eigther maximizes or minimizes a given objective function. A feasible solution that does this is called an optimal solution.

Example: Find the maximum value for the following problem.

→ Given objective function is.

maximize $Z = 3x+4y$

Subj to

$0 \leq x \leq 1$

$-1 \leq y \leq 1$

Assume input set as (x,y)

→ (0,-1), (0,0), (0,1), (1,-1), (1,0), (1,1), (2,2), (3,3) (-4,4)

→ After applying the conditions to input set (2,2) (3,3) (-4,4) pairs are removed from input set. Remaining pairs ~~are~~ {(0,-1) (0,0) (0,1) (1,-1) (1,1)} are called feasible solution.

→ Among these feasible solution, at (1,1) the objective function is maximum

i.e $z = 3x1 + 4x1$

$= 7$

→ So, (1,1) is an optimal solution for the given objective function.

Control Abstraction of Greedy method:

```
Algorithm Greedy (A, n)
// A[1:n] is an input array of size n.
{
    Solution:=0 ; // initialize the solution to empty
    for i:=1 to n do
    {
        x = SELECT(A);
        if FEASIBLE(solution, x) then
        {
            solution := UNION(solution, x);
        }
        else
            reject();
    }
    return solution;
}
```

Applications of Greedy method:

1. Knapsack problem
2. Minimum cost spanning Trees
   - a. Kruskal's Algorithm
   - b. Prim's algorithm
3. Job sequencing with dead lines
4. optimal storage on tapes.

1. Knapsack Problem:

Knapsack is an empty bag. Consider Knapsack whose size is M. It contains n items with weights $w_1, w_2, \ldots w_n$ respectively. The profits of each weight is $p_1, p_2, \ldots p_n$.

→ Let $x_1, x_2, \ldots x_n$ be fractions of items.

→ The problem is to

$$\text{Maximize} \sum_{1 \le i \le n} p_i x_i$$

$$\text{Such That} \sum_{1 \le i \le n} w_i x_i \le M$$

$$\text{and } 0 \le x \le 1,$$

$$\text{and } 1 \le i \le n.$$

→ The profit and weights are positive numbers

Algorithm for Knapsack problem:

```
Algorithm Greedy-Knapsack (M,n)
// W[1:n] is array of weights, P[1:n] is array of Pr
// The objects are arranged in decreasing order
//  such that P[i]/W[i] >= P[i+1]/W[i+1]
// M be the capacity of knapsack
// x[1:n] is solution vector.
{
    for i:=1 to n do x[i]:=0;
     U:= m;
    for i:= 1 to n do
    {
        if (W[i] > U) Then break;
        x[i]:= 1;
        U:= U - W[i];
    }
    if (i<=n) Then
     x[i]:= U/W[i];
}
```

Examples:

① Consider The following instance of knapsack Problem

$$n = 3,$$
$$M = 20$$
$$(P_1, P_2, P_3) = (25, 24, 15)$$
$$(W_1, W_2, W_3) = (18, 15, 10)$$

→ Find The optimal solution for.

ⓐ Maximum profit

ⓑ Minimum weight

ⓒ Maximum profit per unit weight

Sol: Case 1: In This case, we will place an item in The bag whose profit is maximum.

→ $x_1 = 1$, complete item W_1 is kept in The bag i.e, 18 is kept in bag, only 2 units place is left

$$20 - 18 = 2$$

→ x_2 = left out space / weight of item to be placed

$$= 2/15$$

→ $x_3 = 0$, when W_2 is placed no place is left in bag so, $x_3 = 0$, we can't place 3rd item.

$$\sum_{1 \le i \le n} P_i x_i = P_1 x_1 + P_2 x_2 + P_3 x_3$$

$$= 25 \times 1 + 24 \times 2/15 + 15 \times 0$$

$$= 25 + 3.2 + 0$$

$$= 28.2$$

Case 2: In this case we will place an item in bag, whose weight is minimum.

$x_3 = 1.$

$x_2 = 10/15 = 2/3$

$x_1 = 0$

$$\sum P_i x_i = P_1 x_1 + P_2 x_2 + P_3 x_3$$

$$= 25 \times 0 + 24 \times 2/3 + 15 \times 1$$

$$= 0 + 16 + 15$$

$$= 31$$

Case 3: We will place an item in the bag, whose profit per unit weight ratio is maximum.

$$\frac{P_1}{w_1} = \frac{25}{18} = 1.4$$

$$\frac{P_2}{w_2} = \frac{24}{15} = 1.6$$

$$\frac{P_3}{w_3} = \frac{15}{10} = 1.5$$

→ $\frac{P_2}{w_2}$ is maximum so $x_2 = 1$.

Now remaining weight of bag is $20 - 15 = 5$

→ Next maximum is P_3/w_3.

$x_1 = 0$

$x_2 = 1$

$x_3 = 5/10 = 0.5$

$\sum p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3$

$= 0 + 24 \times 1 + 15 \times 0.5$

$= 24 + 7.5$

$= 31.5$

Example 2. Find the optimal solution for Knapsack prob.

where. $n = 7$

$m = 15$

$(p_1, p_2, p_3, p_4, p_5, p_6, p_7) = (10, 5, 15, 7, 6, 18, 3)$

$(w_1, w_2, w_3, w_4, w_5, w_6, w_7) = (2, 3, 5, 7, 1, 4, 1)$

(1) Maximum profit

(2) ~~Maxi~~ minimum weight

(3) Maximum Profit per unit weight.

case 1:

$x_6 = 1$

$x_3 = 1$

$x_1 = 1$

$x_4 = 4/7$, $x_2 = 0$, $x_5 = 0$, $x_7 = 0$

$\sum p_i x_i = p_1 x_1 + p_2 x_2 + p_3 x_3 + p_4 x_4 + p_5 x_5 + p_6 x_6 + p_7 x_7$

$= 10 \times 1 + 0 + 15 \times 1 + 7 \times 4/7 + 0 + 18 \times 1 + 0$

$= 10 + 15 + 4 + 18$

$= 47$

case 2:

$x_7 = 1$

$x_5 = 1$

$x_1 = 1$

$x_2 = 1$

$x_6 = 1$

$x_3 = 4/5$

$$\sum P_i x_i = P_1x_1 + P_2x_2 + P_3x_3 + P_4x_4 + P_5x_5 + P_6x_6 + P_7x_7$$

$$= 10\times1 + 5\times1 + 15\times 4/5 + 6\times1 + 18\times1 + 3\times1$$

$$= 54$$

case 3:

$$\frac{P_1}{w_1} = 10/2 = 5$$

$P_2/w_2 = 5/3 = 1.6$

$P_3/w_3 = 15/5 = 3$

$P_4/w_4 = 7/7 = 1$

$P_5/w_5 = 6/1 = 6$

$P_6/w_6 = 18/4 = 4.5$

$P_7/w_7 = 3/1 = 3$

$$\sum P_i x_i = P_5x_5 + P_1x_1 + P_6x_6 + P_3x_3 + P_7x_7 + P_2x_2$$

$$= 6\times1 + 10\times1 + 18\times1 + 15\times1 + 3\times1 + 5\times 2/3$$

$$= 6 + 10 + 18 + 15 + 3 + 3.3$$

$$= 55.3$$

(2) Minimum Cost Spanning Trees:

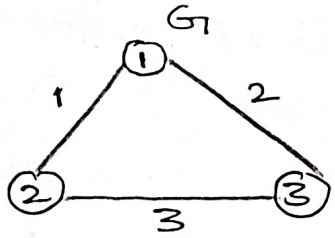
Definition:

Let $G = (V, E)$ be an undirected connected graph. A subgraph $t = (V, E')$ of G is called a spanning tree of G if it satisfies the following conditions.

1) T is a tree without cycles.

2) T contains all vertices of G.

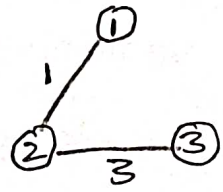
Example:



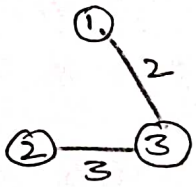
→ If the graph consists of n vertices then the possible spanning trees are n^{n-2},

⇒ From above $n = 3$,

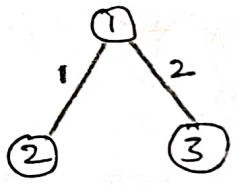
i.e $n^{n-2} = 3^{3-2} = 3$ spanning trees.



cost := 1 + 3 = 4

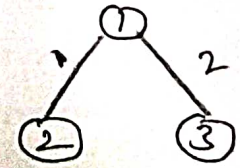


cost := 5



cost := 3

→ For the above graph the minimum cost spanning tree is



→ because it is having minimum cost.

→ similarly, for $n=6$, $6^{6-2} = 6^4 = 1296$, possible spanning trees. It is a time consuming and difficult process as n value increases, so to avoid this difficulty, we use following algorithms

① Prim's algorithm

② Kruskal's algorithm.

① Prim's algorithm:

→ Let $G = (V, E)$ be a graph. Where V is the set of vertices and E is the set of edges. Each edge is associated with a weight.

→ Let S be the solution for minimum spanning tree which is initially empty. Start with some arbitrary vertex i, Select a vertex j such that the cost of edge (i,j) is minimum among the edges that satisfy condition that i is a vertex already included in S and j is a vertex not yet included in S. Include j into the solution such that (i,j) is an edge of minimum cost spanning tree.

→ At each step, an edge with smallest weight connecting a vertex is S to vertex in V-S is added to the tree. It adds only the edge that do not create a cycle.

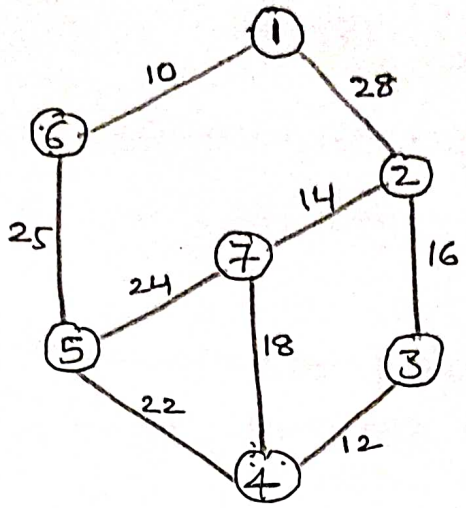
→ when the algorithm terminates, the edge in S form a minimum spanning tree.

```
Algorithm Prim(E, cost, n, t)
// E is the set of edges in G. cost[1:n, 1:n] is the cost
// adjacency matrix of an n vertex graph such that
// cost[i,j] is either a positive real number or ∞
// if no edge (i,j) exists.
// A minimum cost spanning tree is computed and
// stored as a set of edges in the array t[1:n-1,
// t[1:n-1, 1:2], (t[i,1], t[i,2]) is an edge in the
// minimum cost spanning tree. The final cost is returned
{
  // Let (k, l) be an edge of minimum cost in E;
  mincost := cost[k, l];
  t[1,1] := k;
  t[1,2] := l;
  for i := 1 to n do // initialize near.
    if (cost[i, l] < cost[i, k]) then
      near[i] := l;
    else
      near[i] := k;
  near[k] := near[l] := 0;
  for i := 2 to n-1 do
  { // Find n-2 additional edges for t.
    // Let j be an index such that near[j] ≠ 0
    // and cost[j, near[j]] is minimum;
    t[i,1] := j;
    t[i,2] := near[j];
```

```
    mincost := mincost + cost[j, near[j]];
    near[j] := 0;
    for k := 1 to n do // Update near[].
      if ((near[k] ≠ 0) and (cost[k, near[k]] > cost[k,j
        Then near[k] := j;
  }
  return mincost;
}
```

→ The time complexity of prim's algorithm is $O(n^2)$.

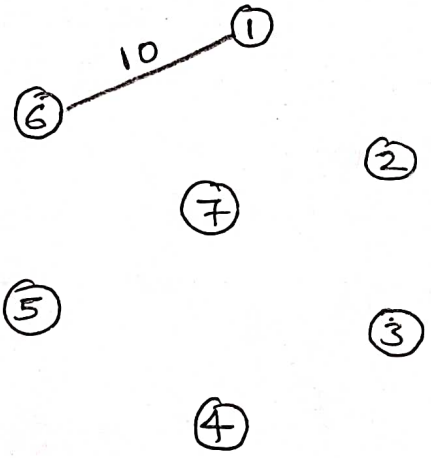
Example: Consider The given graph



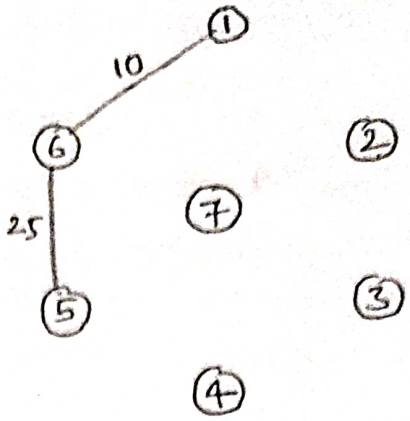
Step1: Initially all vertices are unvisited i-e, S is empty. Assuming Starting vertex as 1

i.e, $S = \{1\}$

→ The edges incident from vertex 1 are (1,2) (1,6) among which minimum cost edge is (1,6) include The edge (1,6).

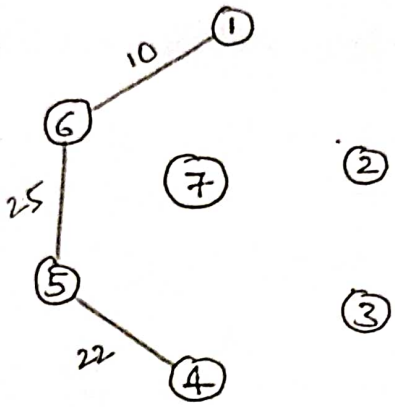


Step 2: Now S contains two vertices 1,6, i-e $S = \{1,6\}$, The edges incident from These vertices are (1,2), (6,5) among which (6,5) is minimum cost edge. Include (6,5) into minimum spanning tree.

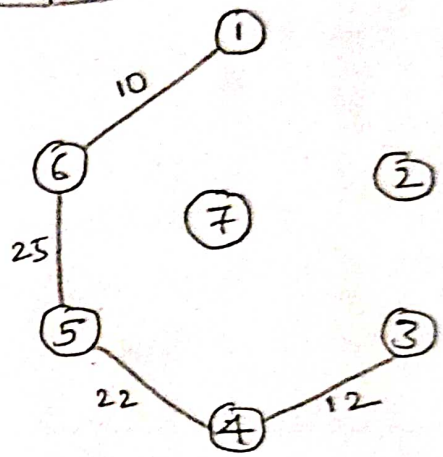


→ Now S = {1,6,5}, repeat ~~This~~ The above procedure until spanning tree contains (n-1) edges.

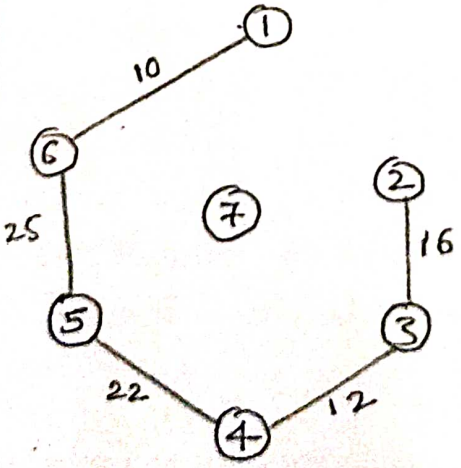
Step3:



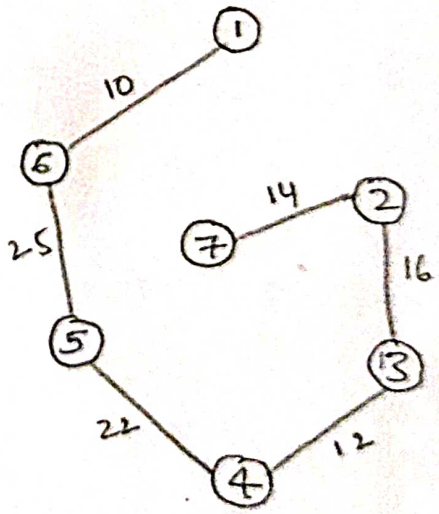
Step4:



Step5:



Step 6:



⇒ ∵ Total cost: = 10+25+22+12+16+14 = 99

② Kruskal's algorithm:

→ Let G be a connected graph consisting of a set of vertices and a set of edges and each edge is associated with a non-negative values. To construct a minimal spanning tree, we use the following procedure.

① Arrange all edges in the increasing order of weight.

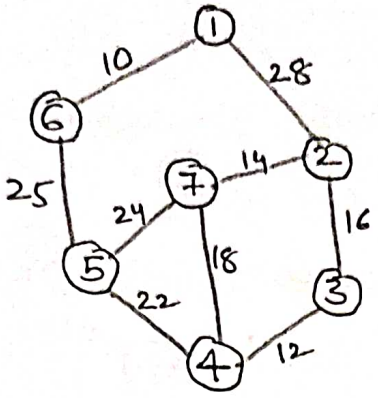
② Select an edge with minimum weight. This is the first edge of spanning tree T to be constructed.

③ Select the next edge with minimum weight that do not form a cycle with the edges already included in T.

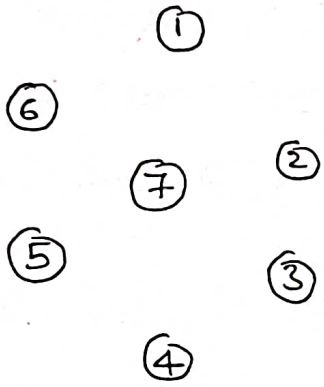
④ Continue step 3 ~~and~~ until T contains (n-1) edges, where n is the number of vertices of G.

```
Algorithm Kruskal(E, cost, n, t)
// E is the set of edges in G. G has n vertices. cost[u,v
// is the cost of edge(u,v). t is the set of edges in
// the minimum-cost spanning tree.
// The final cost is returned.
{ //Construct a heap out of the edge costs using
    Heapify;
    for i:= 1 to n do // Each vertex is in a different set.
    {
        parent[i]:= -1;
    }
    i:=0; mincost:=0.0;
    While((i<n-1) and (heap not empty)) do
    { // Delete a minimum cost edge (u,v) from the he
      and reheapify using Adjust;
      j:= Find(u);
      k:= Find(v);
      if (j≠k) Then
      {
          i:= i+1;
          t[i,1]:= u;
          t[i,2]:= v;
          mincost:= mincost + cost[u,v];
      } Union(j,k);
    }
    if (i≠n-1) Then write("No spanning tree");
    else return mincost;
}
```

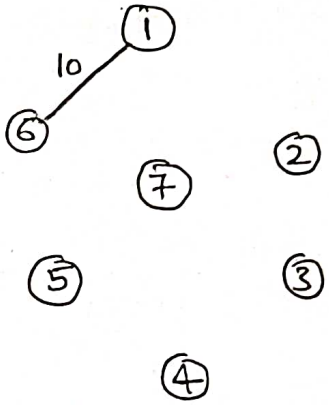
Example: Consider the given graph:



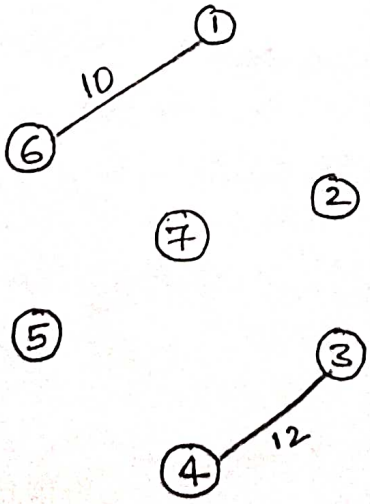
Step 1:



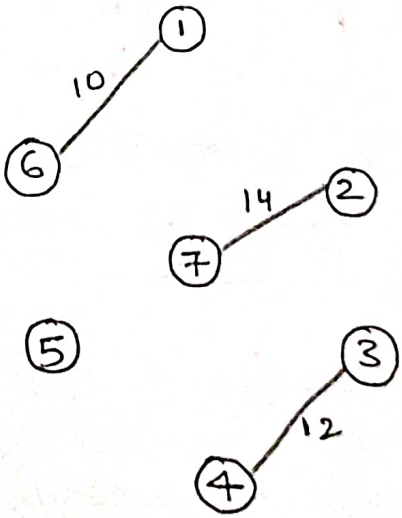
Step 2: select minimum cost among the edge



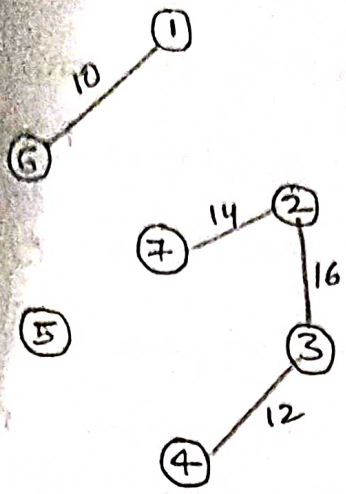
Step 3: select minimum cost among the remaining edges.



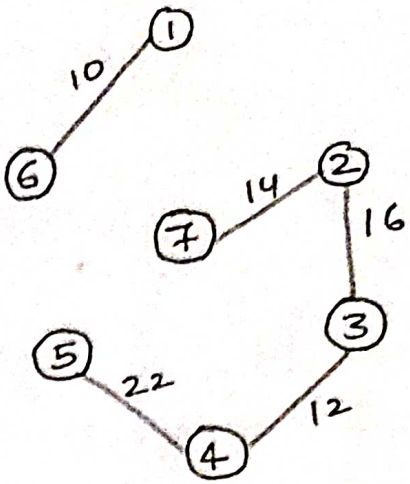
Step 4:



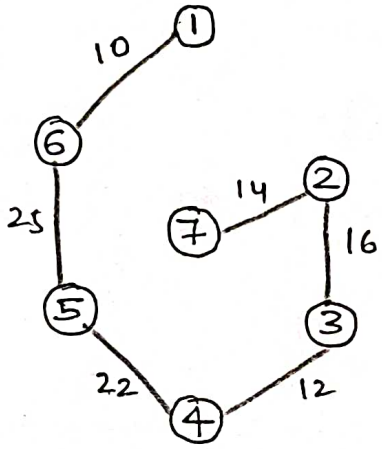
Step 5:



Step 6:



Step 7:



⇒ Total cost = 10 + 25 + 22 + 12 + 14 + 16

= 99

⇒ The computing time of Kruskal's algorithm is

O(E log n) ~~cost~~ ~~O(n log e)~~

→ where E is the number of edges.

③ Optimal storage on tapes:

→ There are n programs that are to be stored on a computer tape of length l. Associated with each program i is a length $l_i, 1 \le i \le n$.

→ clearly, all programs can be stored on the tape if and only if the sum of the lengths of the programs is at most l. We assume that whenever a program is to be retrieved from this tape, the tape is initially positioned at the front.

→ Hence, if the programs are stored in the order $I = i_1, i_2, \ldots i_n$, the time t_j needed to retrieve program i_j is proportional to $\sum_{1 \le k \le j} l_{i_k}$.

→ if all programs are retrieved equally often, then the expected or mean retrieval time (MRT) is

$$(1/n) \sum_{1 \le j \le n} t_j .$$

→ In the optimal storage on tape Problem, we are required to find a permutation for the n programs so that when they are when they are stored on the tape in this order the MRT is minimized.

→ Minimizing the MRT is equivalent to minimizing

$$d(I) = \sum_{1 \le j \le n} \sum_{1 \le k \le j} l_{i_k}$$

```
Algorithm Store(n,m)
// n is the number of programs and m the no. of
{
    j:=0 // Next tape to store on
    for i:=1 to n do
    {   Write("append program", i, "to permutation for
                                    tape", j);
        j:=(j+1) mod m;
    }
}
```

Example: Let $n=3$ and $(l_1, l_2, l_3) = (5, 10, 3)$. There are $n! = 6$ possible orderings. These orderings an. Their respective d values are.

Ordering I	d(I)
1, 2, 3	5+5+10+5+10+3 = 38
1, 3, 2	5+5+3+5+3+10 = 31
2, 1, 3	10+10+5+10+5+3 = 43
2, 3, 1	10+10+3+10+3+5 = 41
3, 1, 2	3+3+5+3+5+10 = 29
3, 2, 1	3+3+10+3+10+5 = 34

⇒ The optimal ordering is

3, 1, 2 = 29

Job sequencing with dead lines:

⇒ If There are $m > 1$ tapes, $T_0, T_1 \ldots T_{m-1}$ Then The programs are

→ The greedy method simply requires us to store The programs in nondecreasing order of Their lengths.

→ From above example The optimal solution is The one in which The programs are in nondecreasing order of Their lengths.

→ if There are $m > 1$ tapes, $T_0, T_1 \ldots T_{m-1}$ Then The programs are to be distributed over These tapes.

$$\text{Total retrieval time (TD)} = \sum_{0 \le j \le m-1} d(I_j)$$

→ if The jobs are initially ordered so That $l_1 \le l_2 \le \ldots \le l_n$, Then The first m programs will be assigned to tapes $T_0, T_1 \ldots T_{m-1}$ respectively. The next m programs will be assigned to tapes $T_0, \ldots T_{m-1}$, respectively.

→ The general rule is That program i is stored on tape $T_{i \bmod m}$.

→ On any given tape The programs are stored in nondecreasing order of Their lengths.

Example: Find an optimal solution placement for 13 programs on three tapes T_0, T_1, and T_2, where the programs are of lengths 12, 5, 8, 32, 7, 5, 18, 26, 4, 3, 11, 10, and 6.

Sol:

Tapes => T_0, T_1 and T_2

Programs => $\overset{1}{12}, \overset{2}{5}, \overset{3}{8}, \overset{4}{32}, \overset{5}{7}, \overset{6}{5}, \overset{7}{18}, \overset{8}{26}, \overset{9}{4}, \overset{10}{3}, \overset{11}{11}, \overset{12}{10}, \overset{13}{6}$

→ arrang programs in increasing order of their le

$\overset{1}{3}, \overset{2}{4}, \overset{3}{5}, \overset{4}{5}, \overset{5}{6}, \overset{6}{7}, \overset{7}{8}, \overset{8}{10}, \overset{9}{11}, \overset{10}{12}, \overset{11}{18}, \overset{12}{26}, \overset{13}{32}$

$n = 3$

$T_0 = 3, 5, 8, 12, 32$

$T_1 = 4, 6, 10, 18$

$T_2 = 5, 7, 11, 26$

$\Rightarrow T_0 = 3+3+5+3+5+8+3+5+8+12+3+5+8+12+32 = 1$

$T_1 = 4+4+6+4+6+10+4+6+10+18 = 72$

$T_2 = 5+5+7+5+7+11+5+7+11+26 = 89$

→ ⊕ $j = 0$.

$T_0 = \phi 3$

$j = 1 \bmod 3 = 1$

$T_1 = 4$

$j = 2 \bmod 3 = 2$

$T_2 = 5$

$j = 3 \bmod 3 = 0$

$T_3 = 5$

Job sequencing with Dead lines

⇒ Job sequencing deals with a set of 'n' jobs, associated with each job 'i' is an integer deadline $d_i \geq 0$ and a profit $P_i \geq 0$. For any job 'i' the profit 'P_i' is earned if the job is completed by its deadline. To complete a job, one have to process the job on a machine for one unit of time.

⇒ A feasible solution for this problem is a subset 'J' of jobs such that each job in this subset can be completed by its deadline.

⇒ The value of the feasible solution 'J' is the sum of the profits of the jobs in 'J', i.e., $\sum_{i \in J} P_i$. An optimal solution is a feasible solution with maximum value.

⇒ TO Obtain the optimal solution using greedy algorithm will depends on how the the next job is choosen.

⇒ The selection of next job has to satisfy the following conditions.

① The job should have maximum profit so as to form an optimal solution.

② The job to be included should be completed by its deadline to form a feasible solution.

Example 1: Solve the job sequencing problem using greedy strategy. Let $n=4$, $(P_1, P_2, P_3, P_4) = (100, 10, 15, 27)$ $(d_1, d_2, d_3, d_4) = (2, 1, 2, 1)$

Sol: Since the maximum deadline time = 2 units, maximum of two jobs will form the feasible solution.

→ According to Greedy strategy, arranging the jobs in decreasing orders of their profits.

$(P_1, P_4, P_3, P_2) = (100, 27, 15, 10)$

$(d_1, d_4, d_3, d_2) = (2, 1, 2, 1)$

	Feasible Solution	Processing Sequence	Profit
1.	(1,2)	2, 1	100+10 = 110
2.	(1,3)	1,3 or 3,1	100+15 = 115
3.	(1,4)	4, 1	100+27 = 127
4.	(2,3)	2, 3	10+15 = 25
5.	(3,4)	4, 3	27+15 = 42
6.	(1)	1	100
7.	(2)	2	10
8.	(3)	3	15
9.	(4)	4	27

⇒ solution 3 is optimal. In this solution only jobs 1 and 4 are processed and the value is 127. These jobs must be processed in the order job4 followed by job1. Thus the processing of job4 begins at time zero and that of job1 is completed at time 2.

```
1. Algorithm JS (d,j,n)
2. // d[i] ≥ 1 , 1 ≤ i ≤ n are the deadlines, n ≥ 1 . The jobs
3. // are ordered such that P[1] ≥ P[2] ≥ ..... ≥ P[n].
4. // J[i] is the ith job in the optimal solution, 1 ≤ i ≤ k.
5. // Also, at termination d[J[i]] ≤ d[J[i+1]], 1 ≤ i < k.
6. {
7.    d[0]:=J[0]:=0;
8.    J[1]:= 1; // include job1.
      k:=1;
      for i:=2 to n do
      {
         // Consider jobs in nonincreasing order of P[i].
         // Find position for i and check feasibility of
         // insertion.
          r:=k;
          while((d[J[r]]>d[i]) and (d[J[r]]≠r)) do
           r:=r-1;
          if((d[J[r]]≤d[i]) and (d[i]>r)) then
          {
              // Insert i into J[].
              for q:=k to (r+1) step-1 do
               J[q+1]:=J[q];
              J[r+1]:= i;
              k:= k+1;
          }
      }
      return k;
}
```

NOTE:

computing time of Js algorithm is $O(n^2)$. it can be reduced to $O(n)$ by using disjoint set union and find algorithms.

Example 2: Given $n=5$,

profits = (1, 5, 20, 15, 10)

deadlines = (1, 2, 4, 1, 3).

Sol:

$(P_1, P_2, P_3, P_4, P_5) = (1, 5, 20, 15, 10)$

Decreasing order

$(P_3, P_4, P_5, P_2, P_1) = (20, 15, 10, 5, 1)$

$(d_3, d_4, d_5, d_2, d_1) = (4, 1, 3, 2, 1)$

S.NO.	Feasible solution	processing	profit
1	{3}	3	20
2	{3,4}	4,3	20+15=35
3	{3,4,5}	4,5,3 or 4,3,5	20+15+10=45
4	{3,4,5,2}	4,2,5,3 or 4,2,3,5	20+15+10+5=50
5	{3,5,2,1}	1,2,5,3	1+5+10+20=36

Optimal solution is 50. Jobs must be processed in the order 4,2,5,3 or 4,2,3,5.

Single Source shortest path problem:

→ Let $G=(V,E)$ be a directed graph with weighting function w for the edges of G.

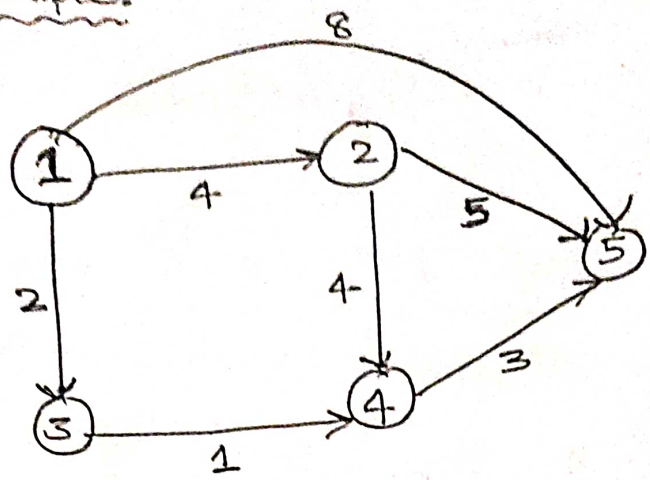
→ The starting vertex of path is called the source and the last vertex is called destination.

→ Let v be the any other vertex which belongs to set of vertices V.

→ The problem to determine a shortest path to given destination vertex v from source is called single source shortest path problem.

⇒ To formulate a greedy based algorithm to generate the shortest paths, we must conceive of a multistage solution to the problem and also of an optimization measure. One possibility is to construct the shortest paths one by one. As an optimization measure, we can use the sum of the lengths of all paths so far generated. This measure is to be minimized.

Example:



Directed Graph.

→ if 1 is the source vertex, then shortest path from 1 to 5 is 6.

→ The shortest paths to other vertices from 1 are.

	Path	Length
1)	1-3	2
2)	1-3-4	3
3)	1-2	4
4)	1-3-4-5	6

→ The greedy method to generate shortest paths from source vertex to the remaining vertices is to generate these paths in increasing order of path length.

⇒ According to Dijkstra's algorithm, first we select a source vertex and include that vertex in the set 'sS'.

→ To generate the shortest paths from source to the remaining vertices a shortest path to the nearest vertex is generated first and it is included in s.

→ Then a shortest path to the second nearest vertex is generated and so on.

→ To generate these shortest paths we need to determine.

1) The next vertex to which a shortest path must be generated and

2) A shortest path to this vertex.

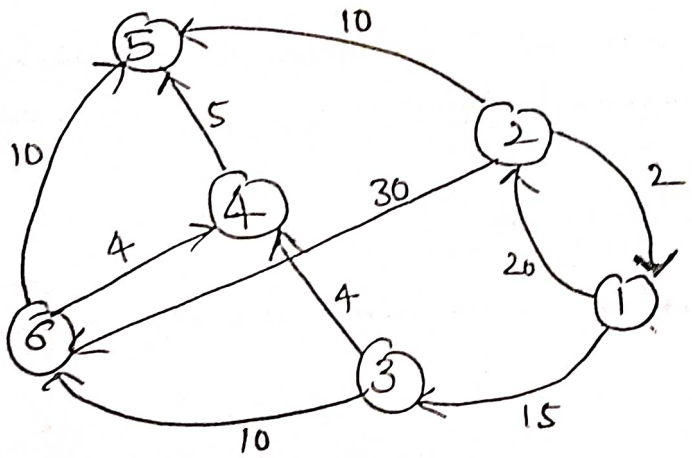
}

```
Algorithm shortest path (G, n, w, dist, V)
// dist[i], 1≤i≤n is the distance or shortest
// Path starting from source passing through
// the vertices that are in S and ending at i.
{
    for i ← 1 to n
    {
        S[i] ← 0 (false);
        dist[i] ← w[v, i];
    }
    S[v] ← 1 (True);
    dist[v] ← 0;
    for k ← 2 to n-1
    {
        choose u from among vertices not in S
        such that dist[u] is minimum
        S[u] ← 1;
        for (each w adjacent to u with S[w]=0)
        {
            if (dist[w] > dist[u] + w[u,w]) then
            dist[w] ← dist[u] + w[u,w];
        }
    }
}
```

The first for loop takes $O(n)$ time. Each execution of second for loop requires $O(n)$ time to select the next vertex and again at the for loop to update dist so that total time for this loop is $O(n^2)$.

Therefore time complexity for this algorithm is $O(n^2)$.

Example: Using shortest path algorithm, obtain in non-decreasing order the length of the shortest path from node one to all the remaining nodes given the following diagraph.



Set S	vertex selected	1	2	3	4	5	6
–	–	0	20	15	∞	∞	∞
{1}	3	0	20	15	19	∞	25
{1,3}	4	0	20	15	19	∞	25
{1,3,4}	2	0	20	15	19	30	25
{1,3,4,2}	6	0	20	15	19	30	25
{1,3,4,2,6}	5	0	20	15	19	30	25

Shortest paths from vertex '1' in increasing order.

1-3 = 15

1-4 = 1-3-4 = 19

1-2 = 20

1-5 = 1-2-5 = 30

1-6 = 1-3-6 = 25

1-3 = 15

1-4 = 1-3-4 = 19

1-2 = 20

1-6 = 1-3-6 = 25

1-5 = 1-2-5 = 30